Practical No. 2

Aim: Symmetric Encryption & Decryption using Python –

a. DES (Data Encryption Standard)
b. AES (Advanced Encryption Standard)

Background Information:

Symmetric encryption in Python utilizes a single shared secret key for both encrypting and decrypting data, offering high efficiency for bulk operations. While the Data Encryption Standard (DES) was a nearly implementation using a 64-bit block size and a weak 56-bit key, it is now considered deprecated and insecure due to vulnerability to brute-force attacks, though still accessible via libraries like PyCryptodome for legacy support. In contrast, the Advanced Encryption Standard (AES) serves as the modern industry standard, employing a larger 128-bit block size and robust key lengths of 128, 192, or 256 bits to ensure high security and performance across various modes (such as CBC, CTR, and GCM) using popular Python libraries like cryptography and PyCryptodome.

1. DES (Data Encryption Standard)

Code:

```
from Crypto.Cipher import DES
def des_encrypt(key, text):
    cipher = DES.new(key, DES.MODE_ECB)
    return cipher.encrypt(text.encode())
def des_decrypt(key, cipher_text):
    cipher = DES.new(key, DES.MODE_ECB)
    return cipher.decrypt(cipher_text).decode().strip()
key = b'mynewkey'
text = "Security"
cipher_text = des_encrypt(key, text)
print("Cipher", cipher_text)
plain_text = des_decrypt(key, cipher_text )
print("Plain Text:", plain_text)
```

Output:

```
Cipher b'Q\x82\xcb\xc3\xa7R\xc0K'
Plain Text: Security
```

2. AES (Advanced Encryption Standard)

Code:

```
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad
def aes_encrypt(key, text):
    cipher = AES.new(key, AES.MODE_ECB)
    return cipher.encrypt(pad(text.encode(), AES.block_size))
def aes_decrypt(key, cipher_text):
    cipher = AES.new(key, AES.MODE_ECB)
    return unpad(cipher.decrypt(cipher_text), AES.block_size).decode()
key = b'mysecretkey12345'
text = "Aryan"
cipher_text = aes_encrypt(key, text)
print("Cipher (hex):", cipher_text.hex())
plain_text = aes_decrypt(key, cipher_text)
print("Plain Text:", plain_text)
```

Output:

```
Cipher (hex): 1cb9860919aaaae60f35f22f45f94896
Plain Text: Aryan
```

Practical No. 3

Aim : Asymmetric Encryption & Decryption using Python
a. RSA
b. Key exchange mechanisms (ECDH)

Background Information :
Asymmetric encryption in Python leverages a public-private key pair to secure communications without pre-sharing secrets, with RSA and ECDH serving distinct but complementary roles. RSA is a robust algorithm used for direct data encryption and digital signatures, relying on large key sizes (typically 2048+ bits) to ensure security, though this makes it computationally intensive. In contrast, ECDH (Elliptic Curve Diffie-Hellman) is optimized for key exchange, allowing two parties to efficiently derive a shared secret over an insecure channel using much smaller key sizes (e.g., 256 bits) while offering forward secrecy. Consequently, modern Python applications often combine these methods by using ECDH to securely negotiate a session key, which is then used for fast symmetric encryption, while reserving RSA for signing or encrypting small data payloads.

1. RSA (Rivest–Shamir–Adleman)

Code:

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
priv = rsa.generate_private_key(65537, 2048)
pub = priv.public_key()
pad = padding.OAEP(padding.MGF1(hashes.SHA256()),
hashes.SHA256(), None)
msg = b"secret data"
cipher = pub.encrypt(msg, pad)
print("Ciphertext:", cipher[:20], "...")
decrypted = priv.decrypt(cipher, pad)
print("Decrypted :", decrypted)
print("Decrypted :", decrypted.decode('utf-8'))
```

Output:

```
Ciphertext: b'`\t\xdd\xda<\x9d\xb5iv[>O\x10\xfb\xc9\xc96b \x85' ...
Decrypted : b'secret data'
Decrypted : secret data
```

2. Key exchange mechanisms (Diffie-Hellman Key Exchange (specifically ECDH - Elliptic Curve Diffie-Hellman)

a. The Successful Exchange (Untampered)

Code:

```
from cryptography.hazmat.primitives.asymmetric import ec
a_priv = ec.generate_private_key(ec.SECP256R1())
b_priv = ec.generate_private_key(ec.SECP256R1())
a_secret = a_priv.exchange(ec.ECDH(), b_priv.public_key())
b_secret = b_priv.exchange(ec.ECDH(), a_priv.public_key())
print("Keys Match?", a_secret == b_secret)
```

Output:

```
Keys Match? True
```

b. The Failed Exchange (Tampered / Intercepted)

Code:

```
from cryptography.hazmat.primitives.asymmetric import ec
a_priv = ec.generate_private_key(ec.SECP256R1())
b_priv = ec.generate_private_key(ec.SECP256R1())
hacker_priv = ec.generate_private_key(ec.SECP256R1())
intercepted_pub_key = hacker_priv.public_key()
a_secret = a_priv.exchange(ec.ECDH(), b_priv.public_key())
b_secret = b_priv.exchange(ec.ECDH(), intercepted_pub_key)
print("Do the shared keys match? :", a_secret == b_secret)
```

Output:

```
Do the shared keys match? : False
```

Practical No. 4

Aim : Hashing and password security

a. MD5
b. SHA
c. Message Authentication Codes (MAC)

Background Information :

In Python, hashing ensures data integrity but requires careful algorithm selection, as MD5 is cryptographically broken and unsafe for any security purpose, while the SHA family (specifically SHA-256 and SHA-3) remains the standard for verifying data integrity and digital signatures. However, neither MD5 nor standard SHA functions are suitable for password storage on their own due to their high speed, which facilitates brute-force attacks; instead, passwords must be hashed using slow, salted algorithms like bcrypt or Argon2. To simultaneously verify data integrity and authenticity, Message Authentication Codes (MAC), particularly HMAC, combine a hash function like SHA-256 with a secret key, ensuring that a message has not been altered and originates from a trusted source.

1. MD5 (Message Digest)

Code:

```
import hashlib
message="Hello, World!"
hashed_message = hashlib.md5(message.encode())
print(hashed_message.hexdigest())
```

Output:

```
65a8e27d8879283831b664bd8b7f0ad4
```

2. SHA (Secure Hash Algorithm)

Code:

```
import hashlib
text=input("Enter the value to encode:")
# Compute
result=hashlib.sha1(text.encode())
# Print hexadecimal digest
print("The hexadecimal equivalent if SHA is: ", result.hexdigest())
```

Output:

```
The hexadecimal equivalent if SHA is:  0a0a9f2a6772942557ab5355d76af442f8f65e01
```

3. MAC (Message Authentication Code)

a. No data tampering:

Code:

```
import hmac,hashlib
# 1. Set up the original message and key
key = b"secret-key"
msg_sent = b"Transfer $100 to Alice"
mac1 = hmac.new(key, msg_sent, hashlib.sha256).hexdigest()
print("Original MAC (mac1)  :", mac1)
# 2. Simulate a safe transmission (No tampering)
msg_received = b"Transfer $100 to Alice"
# 3. Generate the second MAC using the received message
mac2 = hmac.new(key, msg_received, hashlib.sha256).hexdigest()
print("Received MAC (mac2)  :", mac2)
# 4. Compare the two printed MAC strings
is_valid= hmac.compare_digest(mac1,mac2)
print("Is the message valid? :", is_valid)
```

Output:

```
Original MAC (mac1)   : 95b79e8f18e264973861dd02999fa7524c53bc15e0d5680815ad4f92cc4ee69e
Received MAC (mac2)   : 95b79e8f18e264973861dd02999fa7524c53bc15e0d5680815ad4f92cc4ee69e
Is the message valid? : True
```

b. Data tampering:

Code:

```
import hmac, hashlib
# 1. Set up the original message and key
key = b"secret-key"
msg = b"Transfer $100 to Alice"
mac1 = hmac.new(key, msg, hashlib.sha256).hexdigest()
print("Original MAC (mac1) :", mac1)
# 2. Simulate a hacker changing the message
tampered_msg = b"Transfer $9999 to Bob"
# 3. Generate the second MAC using the tampered message
mac2 = hmac.new(key, tampered_msg, hashlib.sha256).hexdigest()
print("Tampered MAC (mac2) :", mac2)
# 4. Compare the two printed MAC strings
is_valid = hmac.compare_digest(mac1, mac2)
# 5. Invert the logic to accurately answer if the data was tampered with
is_tampered = not is_valid
print("Is data tampered?:", is_tampered)
```

Output:

```
Original MAC (mac1) : 95b79e8f18e264973861dd02999fa7524c53bc15e0d5680815ad4f92cc4ee69e
Tampered MAC (mac2) : 80cb38085f7123529f1eddecee3a3dddc6578da3fd00246fde354915440538c6
Is data tampered?: True
```

Practical No. 5

Aim : Digital signature implementation

Background Information :

A Digital Signature is a cryptographic technique used to verify the authenticity, integrity, and non-repudiation of a digital message or document.
A digital signature uses asymmetric (public-key) cryptography, where two keys are used:
Private Key: Used by the sender to create (sign) the digital signature. It is kept secret.
Public Key: Used by the receiver to verify the digital signature. It is shared publicly.

Working:

1. Generate Public and Private Keys using RSA
2. The sender writes the message that needs to be sent.
3. The sender generates a digital signature by signing the message with the private key.
4. The sender sends: The original message and the digital signature
5. The receiver uses the public key to verify the digital signature. 6. Display the Result

Code :

```
import rsa

public_key, private_key = rsa.newkeys(512)

message = "Data Security Practical"

signature = rsa.sign(message.encode(), private_key, 'SHA-256')

print("Digital Signature Created.")

try:
    rsa.verify(message.encode(), signature, public_key)
    print("Signature Verified Successfully.")
except:
    print("Signature Verification Failed.")
```

Output :

```
Digital Signature Created.
Signature Verified Successfully.
```

Practical No. 6

Alm: Demonstration of Firewall Configuration Using Windows Defender Firewall

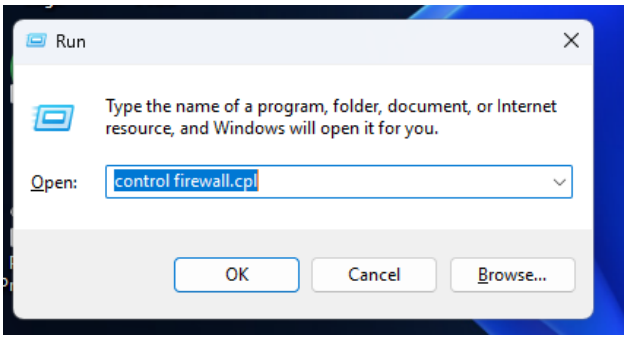
Background Information :

A Firewall is a network security system that monitors and controls Incoming and outgoing network traffic based on predefined security rules.it acts as a barrier between a trusted Internal network and untrusted external networks (such as the Internet).A firewall allows authorized traffic while blocking unauthorized or malicious traffic, thereby protecting computers and networks from cyber threats

Steps to follow:

Step 1

Press Windows + R.

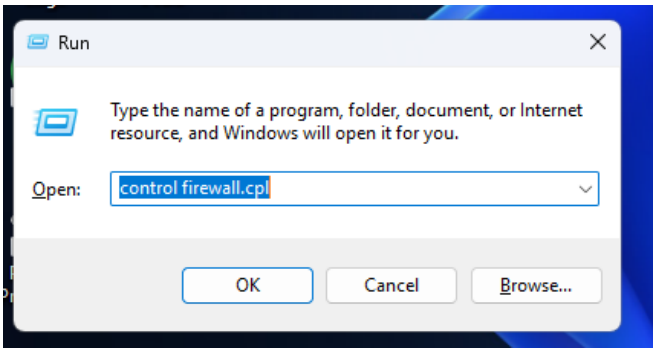


Step 2

Type:

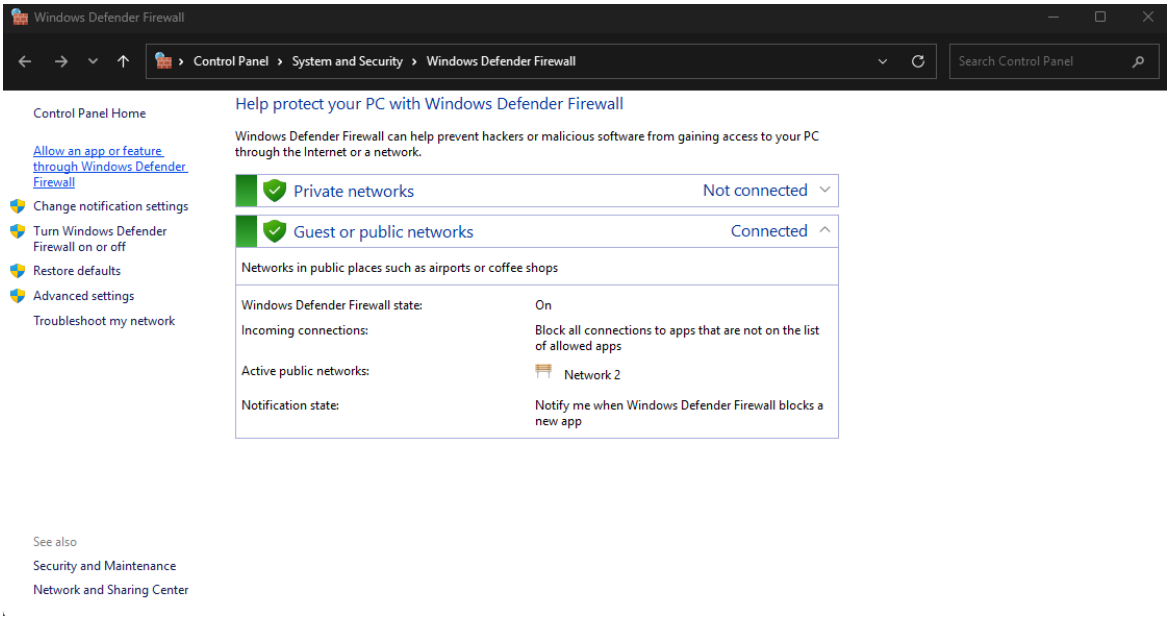
control firewall.cpl

Click OK.



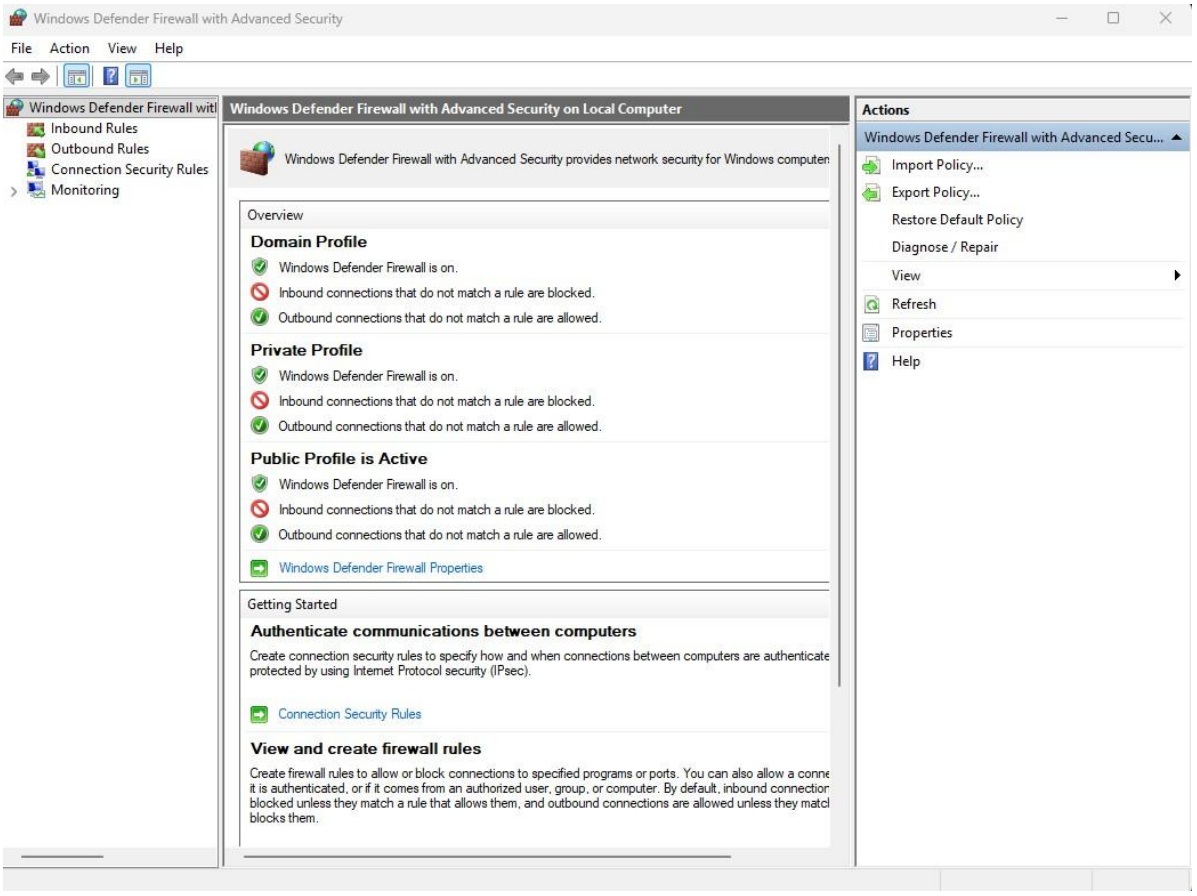
Step 3

Click Advanced Settings.



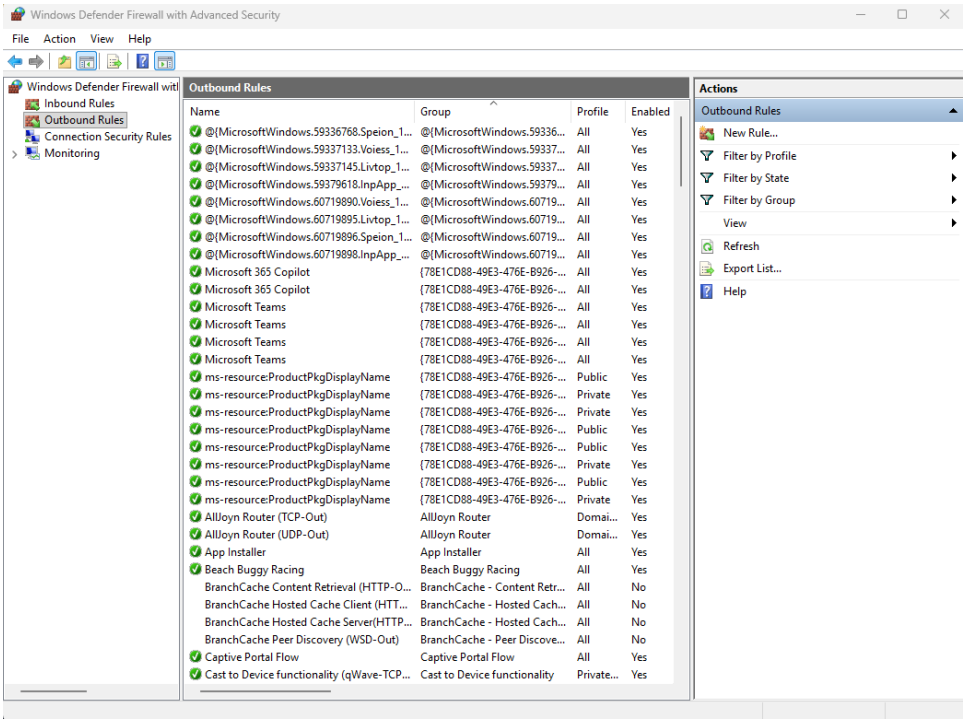
Step 4

Select Outbound Rules (or Inbound Rules)



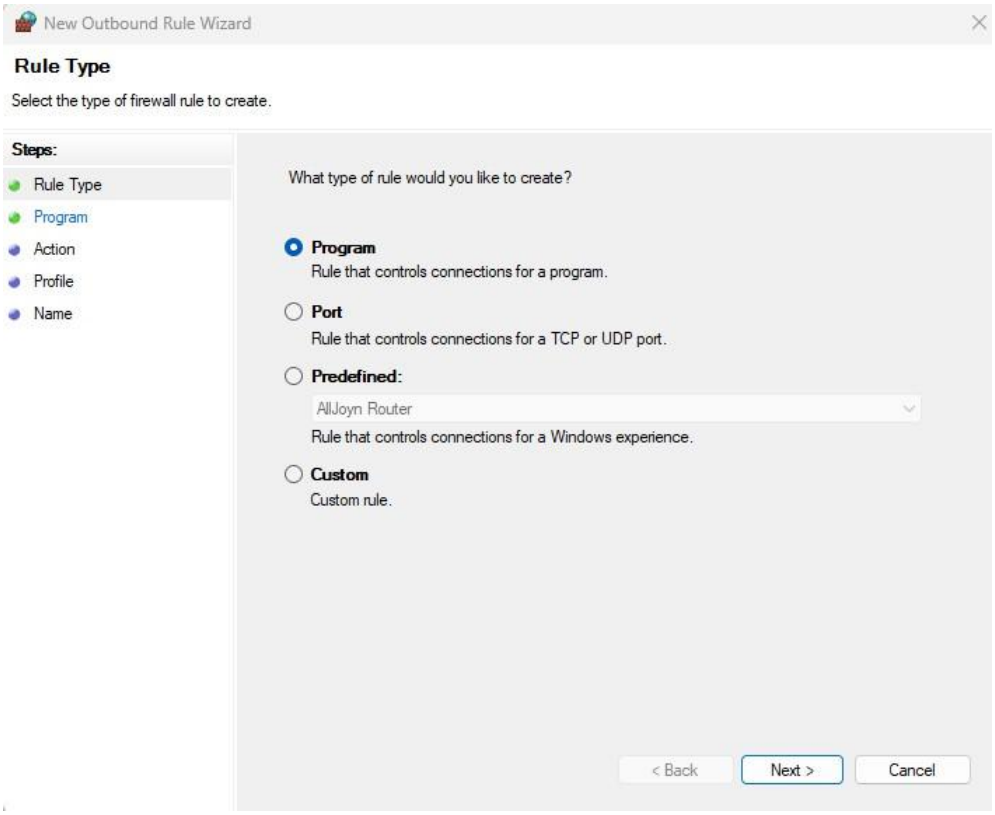
Step 6

Click New Rule.



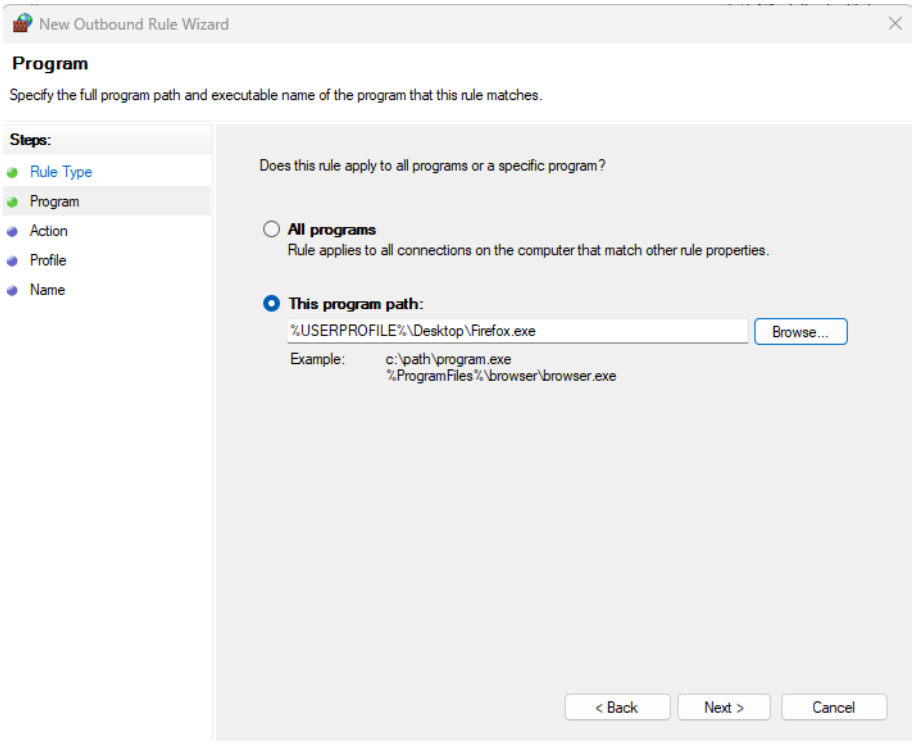
Step 8

Choose Program and click Next.



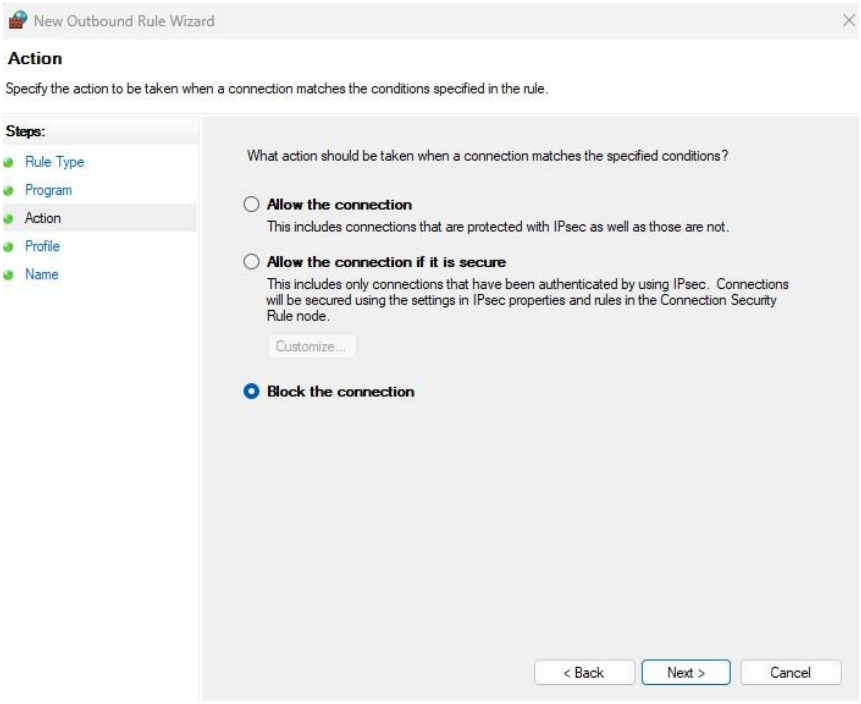
Step 7

Browse and select an application and click Next (for example, Google Chrome in ProgramFlies path).



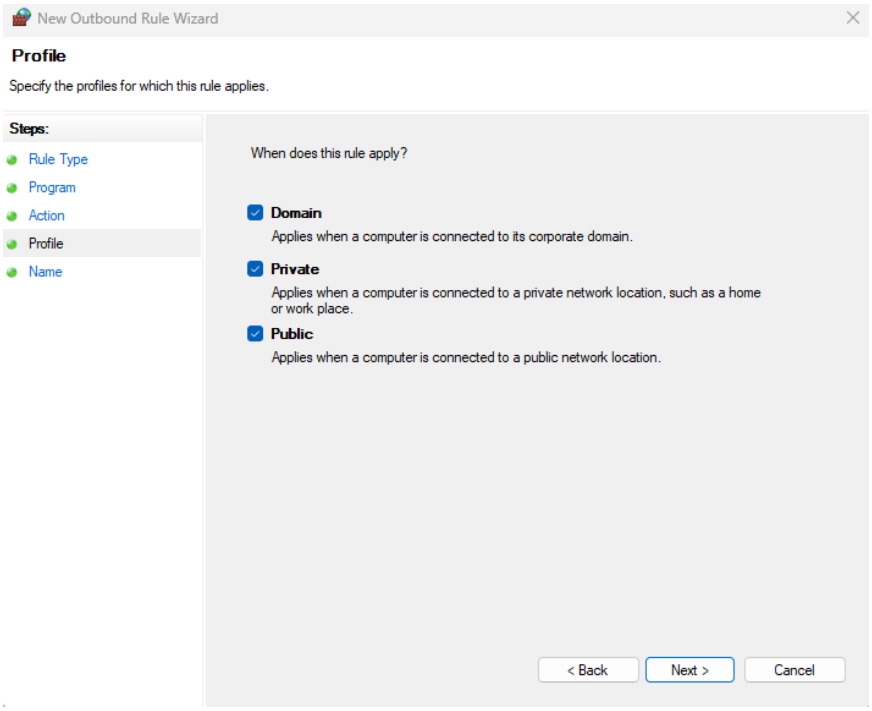
Step 8

Choose Block the Connection.



Step 9

Keep all proflies selected (Domain, Private, Public).



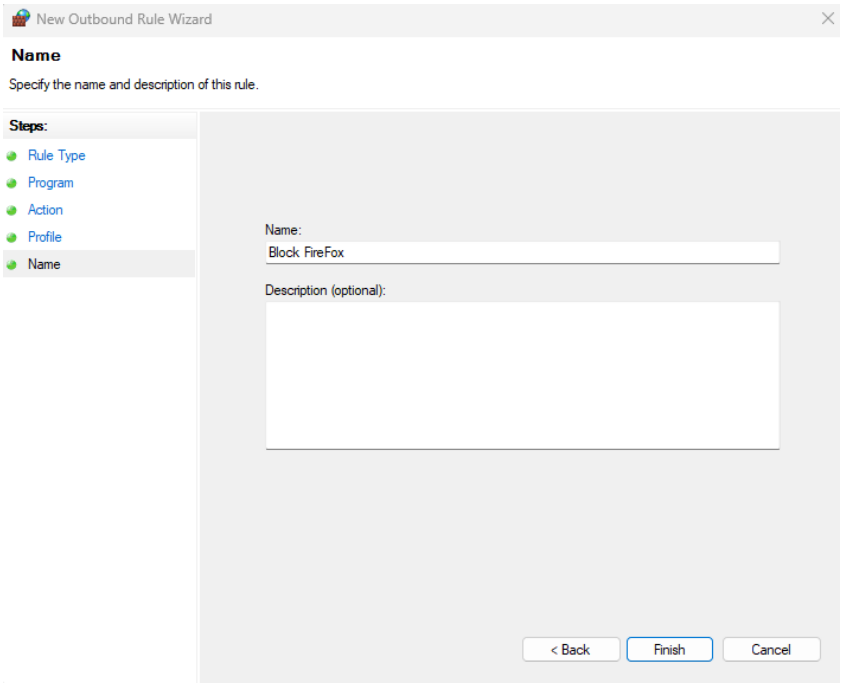
Step 10

Give the rule a name.

Example:

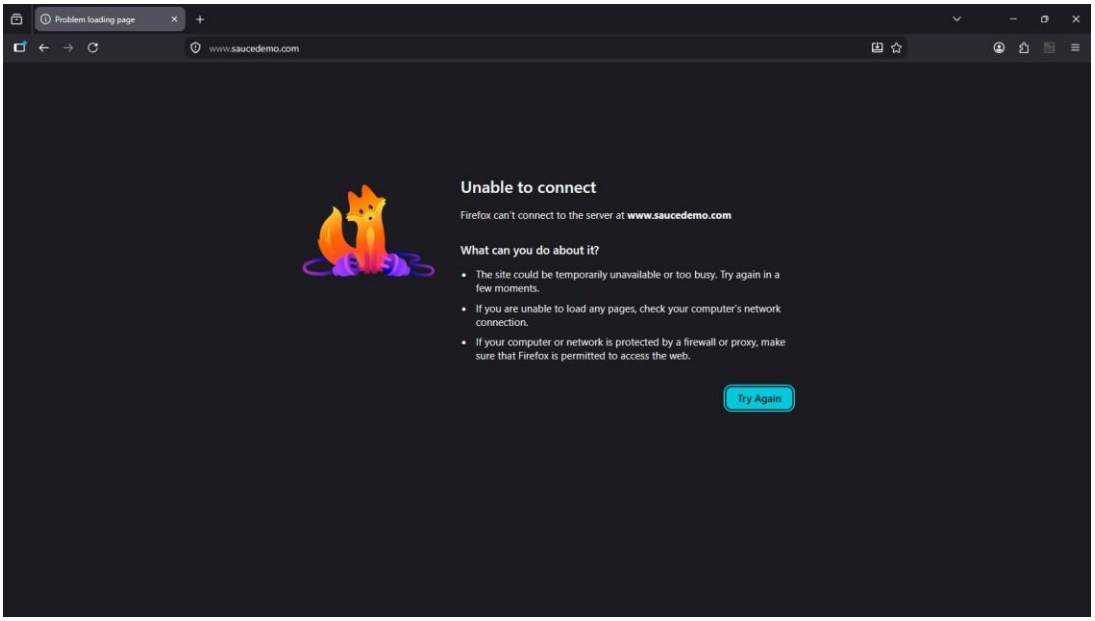
Block Chrome

Click Finish.



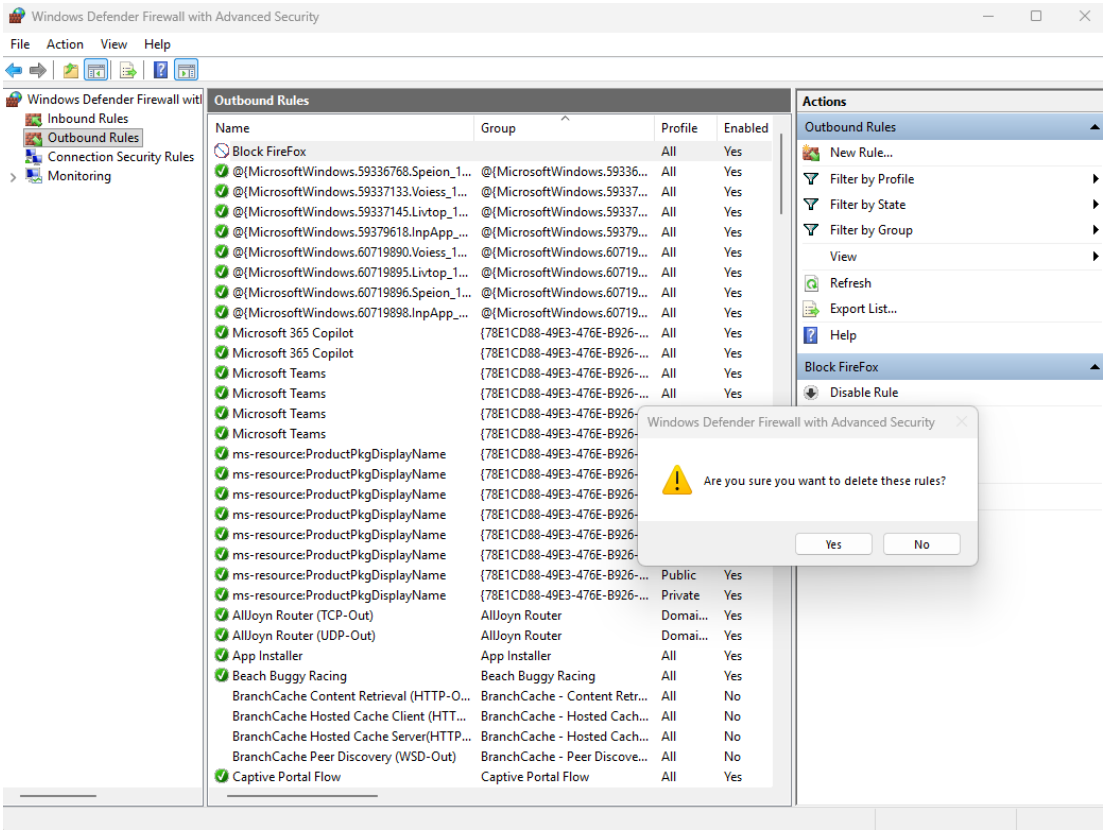
Step 11

Open Chrome and verify that Internet access is blocked.



Step 12

Disable or delete the rule to restore internet access.



PRACTICAL 8

Aim : Vulnerability Scanning Using Nmap on Windows

Background Information :

A vulnerability scan is the process of checking a computer or network for security weaknesses that attackers could exploit.
It helps administrators identify open ports, running services, and potential security risks.
Nmap (Network Mapper) is a free and open-source network scanning tool used to:

- Discover devices on a network.
- Identify open ports.
- Detect running services.
- Gather basic network information.

Steps to follow:

1. Download and install the Nmap Windows Installer from-

https://nmap.org/download.html



2. Verify Nmap Installation:

Open Command Prompt and type:

nmap –version

```
C:\Users\admin>nmap --version
Nmap version 7.991 ( https://nmap.org )
Platform: i686-pc-windows-windows
Compiled with: nmap-liblua-5.4.8 openssl-3.0.21 nmap-libssh2-1.11.1_NMAP1 nmap-libz-1.3.2 nmap-libpcre2-10.47 Npcap-1.88
 nmap-libdnet-1.18.0 ipv6
Compiled without:
Available nsock engines: iocp poll select
```

3. Scan Your Own Computer:

Type:

nmap localhost

```
C:\Users\admin>nmap localhost
Starting Nmap 7.991 ( https://nmap.org ) at 2026-08-31 11:57 +0530
Warning: Hostname localhost resolves to 2 IPs. Using 127.0.0.1.
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00076s latency).
Other addresses for localhost (not scanned): ::1
Not shown: 998 closed tcp ports (reset)
PORT    STATE SERVICE
135/tcp open  msrpc
445/tcp open  microsoft-ds

Nmap done: 1 IP address (1 host up) scanned in 0.46 seconds
```

4. Scan Your Local IP Address

Ipconfig

```
C:\Users\admin>ipconfig

Windows IP Configuration


Ethernet adapter Ethernet:

   Connection-specific DNS Suffix  . : BKBCK.com
   Link-local IPv6 Address . . . . . : fe80::1f86:c8a:91a3:51f1%8
   IPv4 Address. . . . . . . . . . . : 10.200.190.35
   Subnet Mask . . . . . . . . . . . : 255.255.255.128
   Default Gateway . . . . . . . . . : 10.200.190.1

Wireless LAN adapter Local Area Connection* 1:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Wireless LAN adapter Local Area Connection* 2:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::527e:3942:a4fc:77dc%7
   IPv4 Address. . . . . . . . . . . : 192.168.137.1
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . :

Wireless LAN adapter Wi-Fi:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :
```

nmap 192.168.1.15

```
C:\Users\admin>nmap 192.168.1.15
Starting Nmap 7.991 ( https://nmap.org ) at 2026-08-31 11:58 +0530
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 3.45 seconds
```